MONGODB DATABASE OPERATIONS LOG

```
C:\Users\arune>mongosh
Current Mongosh Log ID: 6ae6ca63bcd5e08ac01883ce
Connecting to:          mongodb://127.0.0.1:27017/?
directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.1
4.0
Using MongoDB:          9.0.4
Using Mongosh:          2.14.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/
------
  The server generated these startup warnings when booting
  2026-10-31T18:38:16.090+05:30: Access control is not enabled for the
database. Read and write access to data and configuration is
unrestricted
------

test>
```

Step 1: Connecting to MongoDB Shell

Initial startup and connection to the local MongoDB server.

```
test> use studentDB
switched to db studentDB
studentDB>
```

Step 2: Switching Database

Executing 'use studentDB' to switch to the target database.

```
studentDB> db.createCollection("students")
{ ok: 1 }
studentDB>
```

Step 3: Creating Collection

Creating a new collection named 'students'.

```
studentDB> db.students.insertOne({
  name: "Arun",
  age: 26,
  course: "MERN Stack",
  status: "ongoing"
})
{
  acknowledged: true,
  insertedId: ObjectId('6ae6cafabcd5e08ac01883cf')
}
studentDB>
```

Step 4: Inserting Single Document

Using insertOne() to add the first student record (Arun).

```
studentDB> db.students.insertMany([
  { name: "Kumar", age: 22, course: "MERN Stack", status: "ongoing" },
  { name: "Priya", age: 21, course: "Python", status: "completed" },
  { name: "Divya", age: 24, course: "MERN Stack", status: "completed"
},
  { name: "Rahul", age: 19, course: "Java", status: "ongoing" },
  { name: "Meena", age: 28, course: "Python", status: "ongoing" }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ae6cb17bcd5e08ac01883d0'),
    '1': ObjectId('6ae6cb17bcd5e08ac01883d1'),
    '2': ObjectId('6ae6cb17bcd5e08ac01883d2'),
    '3': ObjectId('6ae6cb17bcd5e08ac01883d3'),
    '4': ObjectId('6ae6cb17bcd5e08ac01883d4')
  }
}
studentDB>
```

Step 5: Inserting Multiple Documents

Using insertMany() to populate the database with several additional student records.

```
studentDB> db.students.find()
[
  { _id: ObjectId('6ae6cafabcd5e08ac01883cf'), name: 'Arun', age: 26,
course: 'MERN Stack', status: 'ongoing' },
  { _id: ObjectId('6ae6cb17bcd5e08ac01883d0'), name: 'Kumar', age: 22,
course: 'MERN Stack', status: 'ongoing' },
  { _id: ObjectId('6ae6cb17bcd5e08ac01883d1'), name: 'Priya', age: 21,
course: 'Python', status: 'completed' },
  { _id: ObjectId('6ae6cb17bcd5e08ac01883d2'), name: 'Divya', age: 24,
course: 'MERN Stack', status: 'completed' },
  { _id: ObjectId('6ae6cb17bcd5e08ac01883d3'), name: 'Rahul', age: 19,
course: 'Java', status: 'ongoing' },
  { _id: ObjectId('6ae6cb17bcd5e08ac01883d4'), name: 'Meena', age: 28,
course: 'Python', status: 'ongoing' }
]
studentDB>
```

Step 6: Retrieving All Records

Using find() to view all documents in the students collection.

```
studentDB> db.students.find({
  age: { $gt: 22 }
})
[
  { _id: ObjectId('6ae6cafabcd5e08ac01883cf'), name: 'Arun', age: 26,
course: 'MERN Stack', status: 'ongoing' },
  { _id: ObjectId('6ae6cb17bcd5e08ac01883d2'), name: 'Divya', age: 24,
course: 'MERN Stack', status: 'completed' },
  { _id: ObjectId('6ae6cb17bcd5e08ac01883d4'), name: 'Meena', age: 28,
course: 'Python', status: 'ongoing' }
]
studentDB>
```

Step 7: Filtering Records

Executing a targeted find() query with a $gt operator (age greater than 22).

```
studentDB> db.students.updateOne(
  { name: "Arun" },
  { $set: { status: "completed" } }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
studentDB>
```

Step 8: Updating a Single Record

Using updateOne() to modify the status of a specific student (Arun).

```
studentDB> db.students.updateMany(
  { status: "ongoing" },
  { $set: { status: "active" } }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}
studentDB>
```

Step 9: Updating Multiple Records

Using updateMany() to change the status of all matching records from 'ongoing' to 'active'.

```
studentDB> db.students.deleteOne({
  name: "Rahul"
})
{ acknowledged: true, deletedCount: 1 }
studentDB>
```

Step 10: Deleting a Record

Using deleteOne() to remove a specific student record (Rahul) from the collection.